Experiment No.9

AIM: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data.

Theory:
1.Dataset Source
Dataset used: Air Quality Dataset (UCI Repository)
Link: https://archive.ics.uci.edu/dataset/360/air+quality

2.Dataset Description
The Air Quality dataset contains environmental sensor data collected in an Italian city.
- Type: Time Series Data
- Instances: ~9,000 records
- Attributes: 15 features

Key Features:

- CO(GT): Carbon monoxide concentration (target variable used)
- PT08.S1(CO): Sensor response
- NMHC(GT): Non-methane hydrocarbons
- NOx(GT): Nitrogen oxides
- Temperature (T)
- Relative Humidity (RH)
- Absolute Humidity (AH)

Target Variable

- The experiment uses CO(GT) (converted to numeric and cleaned)

Data Characteristics

- Contains missing values represented as -200
- Requires preprocessing (cleaning + normalization)
- Sequential/time-dependent → suitable for LSTM

3. Mathematical Formulation of the Algorithm
RNNs process data sequentially. However, Simple RNNs suffer from the "Vanishing Gradient" problem (they forget early inputs). We use LSTM (Long Short-Term Memory) to fix this.
The LSTM Cell:
Unlike a simple neuron, an LSTM cell has an internal "Cell State" (C_t) that acts as a conveyor belt to carry information across time. It uses three "Gates" to regulate this flow:

1.Forget Gate (f_t): Decides what information to throw away from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2.Input Gate (i_t): Decides what new information to store in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

3.Output Gate (o_t): Decides what to output based on the cell state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

4. Algorithm Limitations

- **Sequential Processing:** LSTMs process words one by one. This prevents parallelization (unlike Transformers/BERT), making training slow on long sequences.
- **Long-Term Dependency Cap:** While better than RNNs, LSTMs still struggle with sequences longer than ~500-1000 tokens.
- **Overfitting:** With many parameters (gates), LSTMs easily memorize training data unless regularization (Dropout) is used.

5. Methodology / Workflow

1. Data Loading

    Load CSV file using readtable()

2. Data Cleaning

    Convert column to numeric
    Replace -200 with NaN
    Remove missing values

3. Normalization

    Normalize data for better training

4. Sequence Creation

    Input: XtX_tXt
    Output: Xt+1X_{t+1}Xt+1

5. Model Design

    Sequence Input Layer
    LSTM Layer (50 units)
    Fully Connected Layer
    Regression Layer

2.Training

    Optimizer: Adam
    Epochs: 50

3. Prediction

    Predict values using trained model

4. Visualization

    Plot Actual vs Predicted values

6. Performance Analysis

    Training completed for 50 epochs

    RMSE values fluctuate between ~0.5 to 1.3 (page 3–4 logs)

Loss reduces gradually → model is learning

6. Hyperparameter Tuning

Parameters Used:

- LSTM Units = 50
- Epochs = 50
- Optimizer = Adam
- Learning Rate ≈ 0.001

Tuning Strategies

Parameter	Effect
Increase LSTM units	Better learning but more computation
Increase epochs	Improves accuracy but risk of overfitting
Learning rate	Too high → unstable, too low → slow learning
Batch size	Affects convergence speed

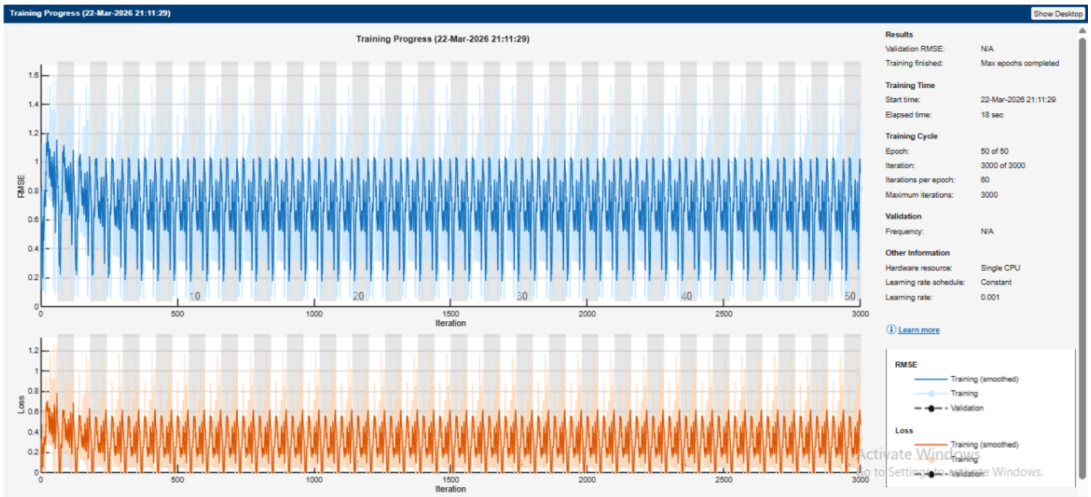
Possible Improvements

- Increase LSTM units (e.g., 100)
- Add Dropout layer to prevent overfitting
- Use multiple LSTM layers (Deep LSTM)
- Try Bidirectional LSTM

7. Code and Output

```
% Load dataset
data = readtable('AirQualityUCI.csv');

% Convert column to numeric (IMPORTANT
FIX) series = str2double(data{:,3});


% Replace invalid values (-200)
series(series == -200) = NaN;


% Remove missing values
series = series(~isnan(series));


% Normalize series =
normalize(series);
```

```
% Create sequences
   X        = series(1:end-1);
   Y        = series(2:end);


XTrain = num2cell(X');
YTrain = num2cell(Y');


% LSTM network
layers = [
sequenceInputLayer(1)
lstmLayer(50)
fullyConnectedLayer(1)
regressionLayer
];


% Training options options =
trainingOptions('adam', ...
          'MaxEpochs', 50, ...
          'Plots','training-progress');


% Train net = trainNetwork(XTrain, YTrain,
layers, options);


% Predict
YPred = predict(net, XTrain);


YPred = cell2mat(YPred);
YTrue = cell2mat(YTrain);


% Plot results figure; plot(YTrue,
'b'); hold on; plot(YPred, 'r');
legend('Actual','Predicted');
title('LSTM Time Series
Prediction');
```



Command Window

```
Training on single CPU.
|=========================================================================================|
| Epoch | Iteration | Time Elapsed | Mini-batch | Mini-batch | Base Learning |
|       |           | (hh:mm:ss)   | RMSE       | Loss       | Rate          |
|=========================================================================================|
|     1 |         1 |     00:00:02 |       0.82 |        0.3 |        0.0010 |
|     1 |        50 |     00:00:03 |       0.71 |        0.2 |        0.0010 |
|     2 |       100 |     00:00:03 |       1.30 |        0.8 |        0.0010 |
|     3 |       150 |     00:00:04 |       0.82 |        0.3 |        0.0010 |
|     4 |       200 |     00:00:04 |       1.35 |        0.9 |        0.0010 |
|     5 |       250 |     00:00:04 |       0.47 |        0.1 |        0.0010 |
|     5 |       300 |     00:00:04 |       0.81 |        0.3 |        0.0010 |
|     6 |       350 |     00:00:05 |       0.32 |    5.0e-02 |        0.0010 |
|     7 |       400 |     00:00:05 |       1.19 |        0.7 |        0.0010 |
|     8 |       450 |     00:00:05 |       0.81 |        0.3 |        0.0010 |
|     9 |       500 |     00:00:05 |       1.33 |        0.9 |        0.0010 |
|    10 |       550 |     00:00:06 |       0.53 |        0.1 |        0.0010 |
|    10 |       600 |     00:00:06 |       0.81 |        0.3 |        0.0010 |
|    11 |       650 |     00:00:06 |       0.31 |    4.7e-02 |        0.0010 |
|    12 |       700 |     00:00:06 |       1.19 |        0.7 |        0.0010 |
|    13 |       750 |     00:00:07 |       0.81 |        0.3 |        0.0010 |
|    14 |       800 |     00:00:07 |       1.33 |        0.9 |        0.0010 |
|    15 |       850 |     00:00:07 |       0.53 |        0.1 |        0.0010 |
|    15 |       900 |     00:00:08 |       0.81 |        0.3 |        0.0010 |
|    16 |       950 |     00:00:08 |       0.31 |    4.7e-02 |        0.0010 |
|    17 |      1000 |     00:00:08 |       1.19 |        0.7 |        0.0010 |
|    18 |      1050 |     00:00:08 |       0.81 |        0.3 |        0.0010 |
|    19 |      1100 |     00:00:09 |       1.33 |        0.9 |        0.0010 |
|    20 |      1150 |     00:00:09 |       0.53 |        0.1 |        0.0010 |
|    20 |      1200 |     00:00:09 |       0.81 |        0.3 |        0.0010 |
|    21 |      1250 |     00:00:09 |       0.31 |    4.7e-02 |        0.0010 |
|    22 |      1300 |     00:00:10 |       1.19 |        0.7 |        0.0010 |
|    23 |      1350 |     00:00:10 |       0.81 |        0.3 |        0.0010 |
```

Command Window

27	1600	00:00:11	1.19	0.7	0.0010
28	1650	00:00:11	0.81	0.3	0.0010
29	1700	00:00:12	1.33	0.9	0.0010
30	1750	00:00:12	0.53	0.1	0.0010
30	1800	00:00:12	0.81	0.3	0.0010
31	1850	00:00:12	0.31	4.7e-02	0.0010
32	1900	00:00:13	1.19	0.7	0.0010
33	1950	00:00:13	0.81	0.3	0.0010
34	2000	00:00:13	1.33	0.9	0.0010
35	2050	00:00:13	0.53	0.1	0.0010
35	2100	00:00:14	0.81	0.3	0.0010
36	2150	00:00:14	0.31	4.7e-02	0.0010
37	2200	00:00:14	1.19	0.7	0.0010
38	2250	00:00:14	0.81	0.3	0.0010
39	2300	00:00:15	1.33	0.9	0.0010
40	2350	00:00:15	0.53	0.1	0.0010
40	2400	00:00:15	0.81	0.3	0.0010
41	2450	00:00:15	0.31	4.7e-02	0.0010
42	2500	00:00:16	1.19	0.7	0.0010
43	2550	00:00:16	0.81	0.3	0.0010
44	2600	00:00:16	1.33	0.9	0.0010
45	2650	00:00:16	0.53	0.1	0.0010
45	2700	00:00:17	0.81	0.3	0.0010
46	2750	00:00:17	0.31	4.7e-02	0.0010
47	2800	00:00:17	1.19	0.7	0.0010
48	2850	00:00:17	0.81	0.3	0.0010
49	2900	00:00:18	1.33	0.9	0.0010
50	2950	00:00:18	0.53	0.1	0.0010
50	3000	00:00:18	0.81	0.3	0.0010

Training finished: Max epochs completed.

Conclusion

The LSTM model successfully performs time-series forecasting on air quality data. It captures temporal dependencies effectively, though performance can be further improved with better tuning and model complexity.